

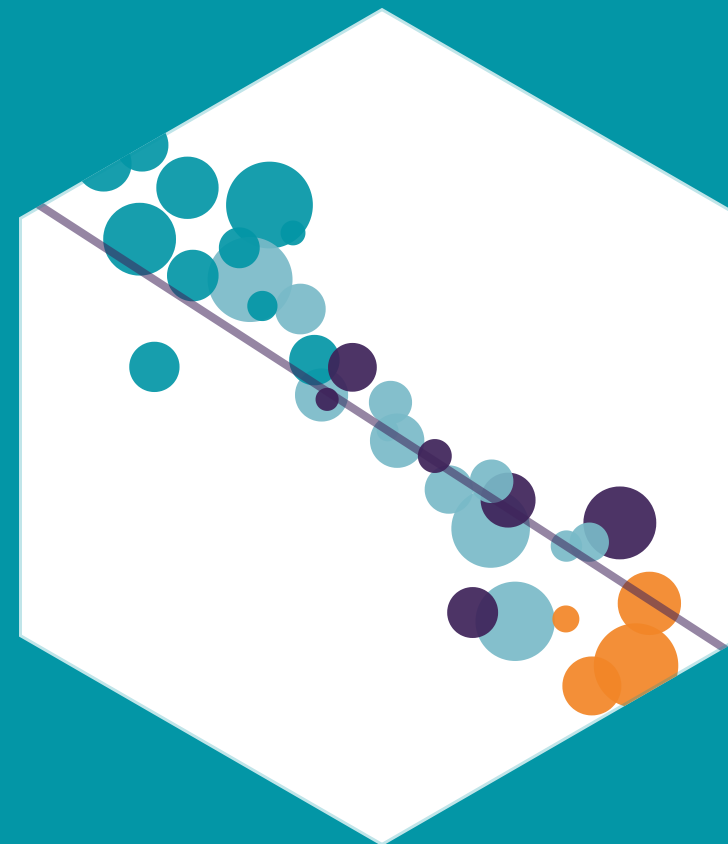
ECONOMETRICS

TA SESSION 3

Barcelona School of Economics

Lucia Sauer

✉ lucia.sauer@bse.eu



Overview

- Monte Carlo experiments: the sampling distribution of OLS
- How the DGP shapes that distribution
- A full regression output: estimates, standard errors, t -values, p -values
- Regression tables with `stargazer` and export to LaTeX

Why These Topics?

🎲 Monte Carlo Experiments

- $\hat{\beta}$ is a **random variable**: a new sample gives a new estimate
- Simulation lets us *see* its distribution when we know the true DGP

⚙️ Changing the DGP

- Noise, sample size, collinearity: what makes an estimator **precise**?

📊 Regression Output

- Every number in a regression table is a statistic of that sampling distribution

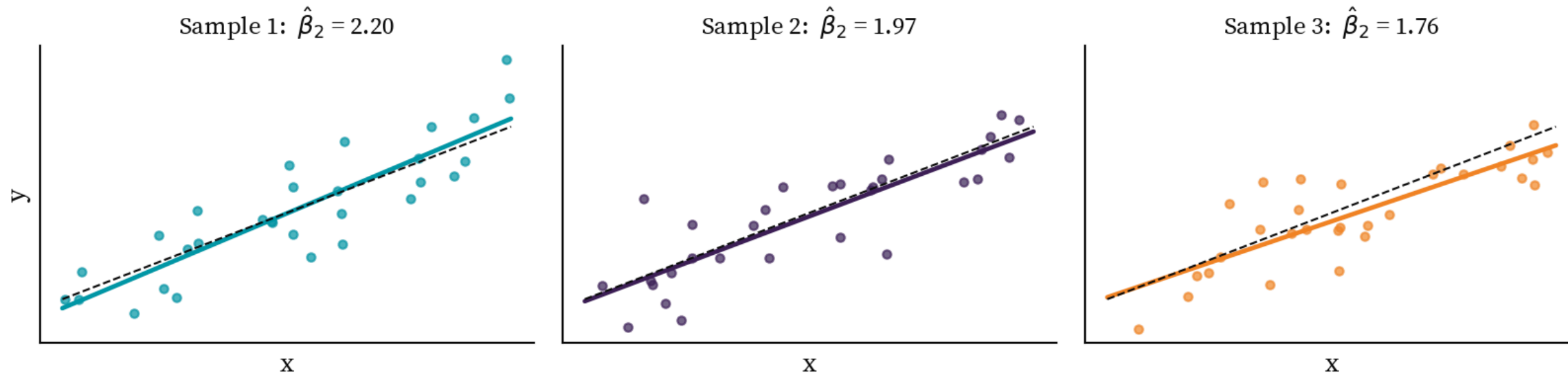
Monte Carlo Experiments

An estimator is a random variable

In practice we observe **one** sample and compute **one** estimate:

$$\hat{\beta} = (X'X)^{-1} X'y$$

But the sample is random: a **different sample** from the same population gives a **different** $\hat{\beta}$.



Dashed line: true relation, slope $\beta_2 = 2$

The sampling distribution

The **sampling distribution** of $\hat{\beta}$ is its distribution across **all possible samples** of size n drawn from the same DGP. It tells us two things:

- **Centre:** is $\hat{\beta}$ **unbiased**, $E(\hat{\beta}) = \beta$?
- **Spread:** how **precise** is it, $Var(\hat{\beta})$?

The problem: we never see it. With real data we have a single sample, and the true β is unknown.

The Monte Carlo idea

Play the role of *nature*: **choose** the DGP (so we know the true β), draw many samples from it on the computer, and look at the estimates.

Seeing it: the sample mean

1 · POPULATION

Choose a probability distribution: the population (DGP) we sample from.

Normal

▼

2 · ONE SAMPLE

Choose a sample size n and sample once.

$n = 10$

Sample

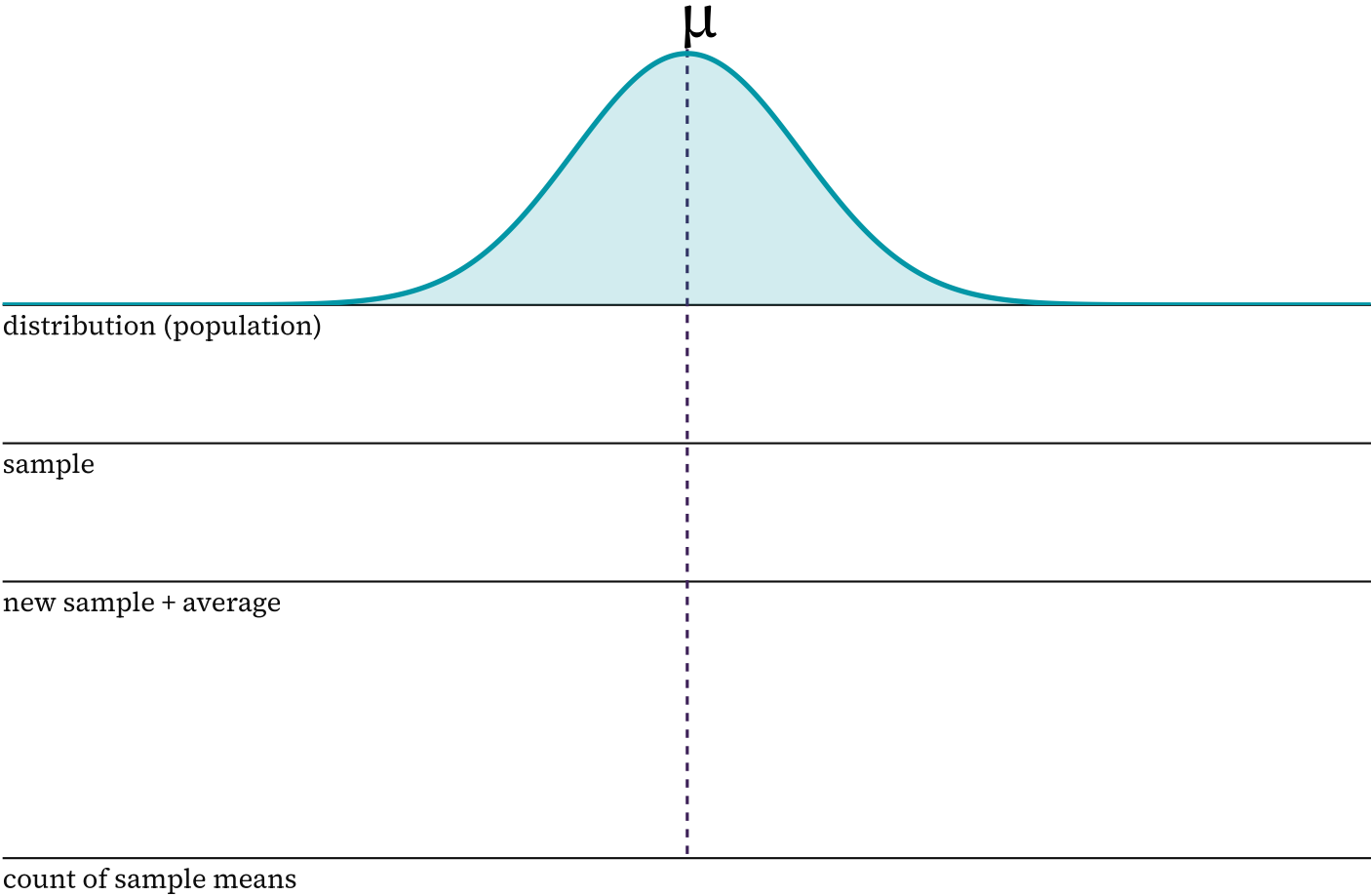
3 · REPEAT THE EXPERIMENT

Draw new samples from the population and average each one.

New sample

100 new samples

Adapted from [Seeing Theory](#) (D. Kunin et al., Brown University).



What does the Monte Carlo experiment show?

The steps we just followed

- 1 Fix a **population** we know (here: its mean μ)
- 2 Draw a random **sample** of size n
- 3 Compute the **statistic**: $\bar{x}$
- 4 **Repeat** steps 2–3 many times
- 5 Plot the **histogram** of all the $\bar{x}$'s

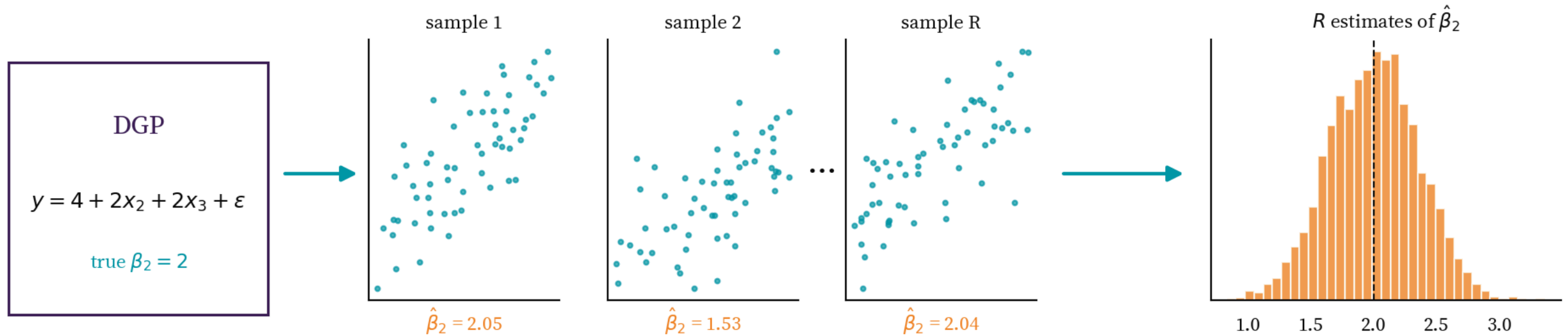
What we see

The histogram approximates the **sampling distribution** of $\bar{x}$:

- **Centre** at μ : $\bar{x}$ is unbiased
- **Spread** shrinks with n : $sd(\bar{x}) = \sigma/\sqrt{n}$
- **Shape** close to normal for large n , even if the population is skewed (CLT)

Monte Carlo for an estimator

Same experiment, but the population is a regression DGP and, instead of $\bar{x}$, each sample gives us the OLS estimate $\hat{\beta}_2$



1 Specify a DGP

parameters, regressors,
errors

2 Generate

one sample of size n

3 Estimate

$\hat{\beta}_2$ by OLS, store it

4 Repeat

steps 2-3, $R = 10,000$
times

5 Summarise

histogram / KDE: centre,
spread, shape

Data-Generating Process (DGP)

We generate $R = 10,000$ samples of size $n = 100$ from:

$$y_i = 4 + 2x_{i2} + 2x_{i3} + \varepsilon_i, \quad \varepsilon_i \mid X \stackrel{iid}{\sim} N(0, \sigma_\varepsilon^2)$$
$$x_{i2} \sim U[0, 40], \quad x_{i3} = x_{i2} + v_i, \quad v_i \sim N(0, \sigma_v^2)$$

- Baseline values: $\sigma_\varepsilon = 32$, $\sigma_v = 16$
- True parameter of interest: $\beta_2 = 2$
- x_{i3} is **correlated** with x_{i2} through v_i

Question

Under this DGP, is $\hat{\beta}_2$ unbiased? What should the histogram be centred at?

Monte Carlo in code

```
1 def simulate_betas(n=100, sigma_eps=32, sigma_v=16, x2_max=40, reps=10_000, seed=42):
2     rng = np.random.default_rng(seed)
3     betas = np.empty(reps)
4     for r in range(reps):
5         # 2. generate one sample from the DGP
6         x2 = rng.uniform(0, x2_max, n)
7         x3 = x2 + rng.normal(0, sigma_v, n)
8         eps = rng.normal(0, sigma_eps, n)
9         y = 4 + 2 * x2 + 2 * x3 + eps
10        # 3. estimate by OLS,  $(X'X)^{-1} X'y$ , and keep beta_2
11        X = np.column_stack([np.ones(n), x2, x3])
12        betas[r] = np.linalg.solve(X.T @ X, X.T @ y)[1]
13    return betas
```

Note

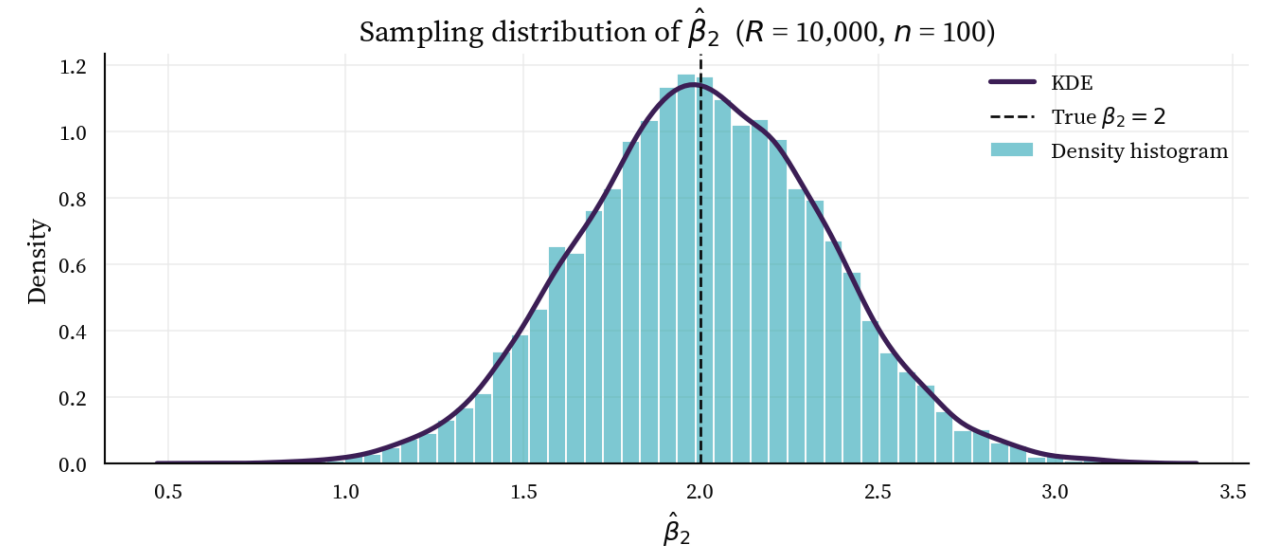
`sm.OLS(y, X).fit()` gives the same $\hat{\beta}$, but solving $(X'X)\hat{\beta} = X'y$ directly is much faster inside a 10,000-iteration loop.

Summarising the 10,000 estimates

CODE

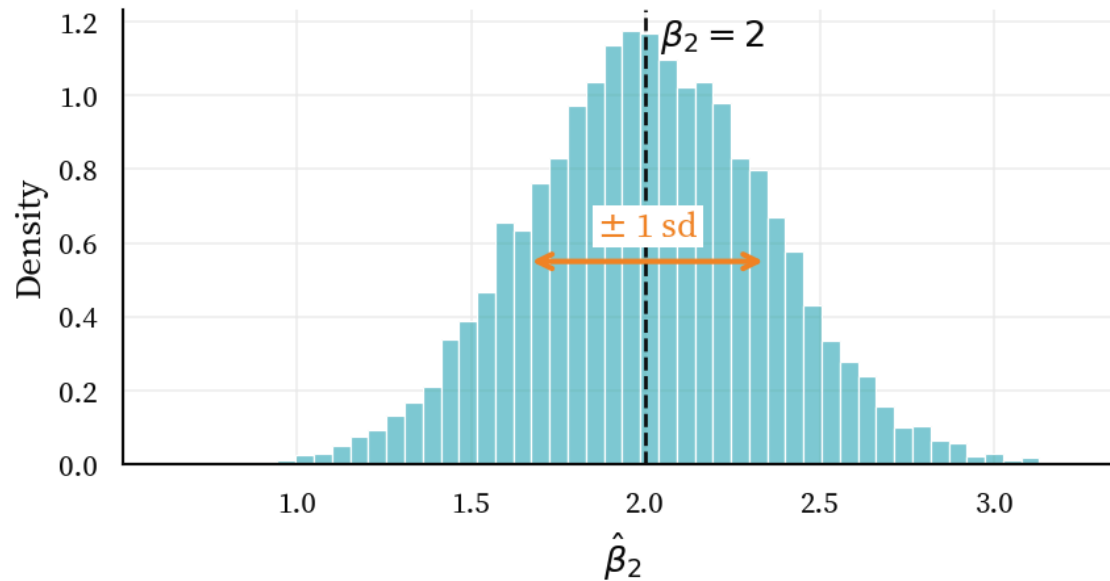
```
1 betas_base = simulate_betas()
2
3 fig, ax = plt.subplots(figsize=(9, 4.2))
4 sns.histplot(betas_base, bins=50, stat="density", color=bse_style.TEAL, alpha=0.5, ax=ax)
5 sns.kdeplot(betas_base, color=bse_style.PURPLE, linewidth=2.5, ax=ax)
6 ax.axvline(2, color="black", ls="--", label=r"True $\beta_2$")
7 plt.show()
8
9 print(f"mean = {betas_base.mean():.4f}, sd = {betas_base.std():.4f}")
```

OUTPUT



mean = 2.0056, sd = 0.3477

Reading the histogram



- **Centre:** the mean of the 10,000 estimates is ≈ 2 , so $\hat{\beta}_2$ is **unbiased**
- **Spread:** the sd of the 10,000 estimates ≈ 0.35 tells us how much $\hat{\beta}_2$ moves from sample to sample
- **Shape:** bell-shaped, as expected with normal errors

Exercise

What fraction of the 10,000 estimates lies within $\beta_2 \pm 1.96 \cdot \text{sd}$? Compute it with `np.mean(np.abs(betas_base - 2) < 1.96 * betas_base.std())`.

Why is it close to **95%**? This is the logic behind **confidence intervals** and **t-tests**: we will use it in the coming sessions.

Standard deviation vs. standard error

Standard deviation of the estimator

$$sd(\hat{\beta}_2 | X) = \sqrt{\sigma_\varepsilon^2 [(X'X)^{-1}]_{22}}$$

A **population** quantity: depends on the unknown σ_ε^2 .
With a Monte Carlo we can approximate it by the sd of the 10,000 estimates.

Standard error

$$se(\hat{\beta}_2) = \sqrt{\hat{\sigma}^2 [(X'X)^{-1}]_{22}}, \quad \hat{\sigma}^2 = \frac{SSE}{n-K}$$

An **estimate** of that sd, computed from **one sample**: it is what the regression output reports in the `std err` column.

```
1 import statsmodels.api as sm
2
3 rng = np.random.default_rng(1) # ONE sample from the same DGP
4 x2 = rng.uniform(0, 40, 100); x3 = x2 + rng.normal(0, 16, 100)
5 y = 4 + 2 * x2 + 2 * x3 + rng.normal(0, 32, 100)
6 one_sample = sm.OLS(y, sm.add_constant(np.column_stack([x2, x3]))).fit()
7
8 print(f"Monte Carlo sd of beta_2_hat (10,000 samples): {betas_base.std():.3f}")
9 print(f"Standard error from one sample: {one_sample.bse[1]:.3f}")
```

```
Monte Carlo sd of beta_2_hat (10,000 samples): 0.348
Standard error from one sample: 0.322
```

Changing the DGP

What drives the spread of $\hat{\beta}_2$?

Under homoskedasticity, the variance of the OLS estimator (conditional on X) is

$$\text{Var}(\hat{\beta}_2 \mid X) = \sigma_\varepsilon^2 \left[(X'X)^{-1} \right]_{22} = \frac{\sigma_\varepsilon^2}{\sum_{i=1}^n (x_{i2} - \bar{x}_2)^2 (1 - R_2^2)}$$

where R_2^2 is the R^2 of regressing x_2 on the other regressors (x_3 and a constant).

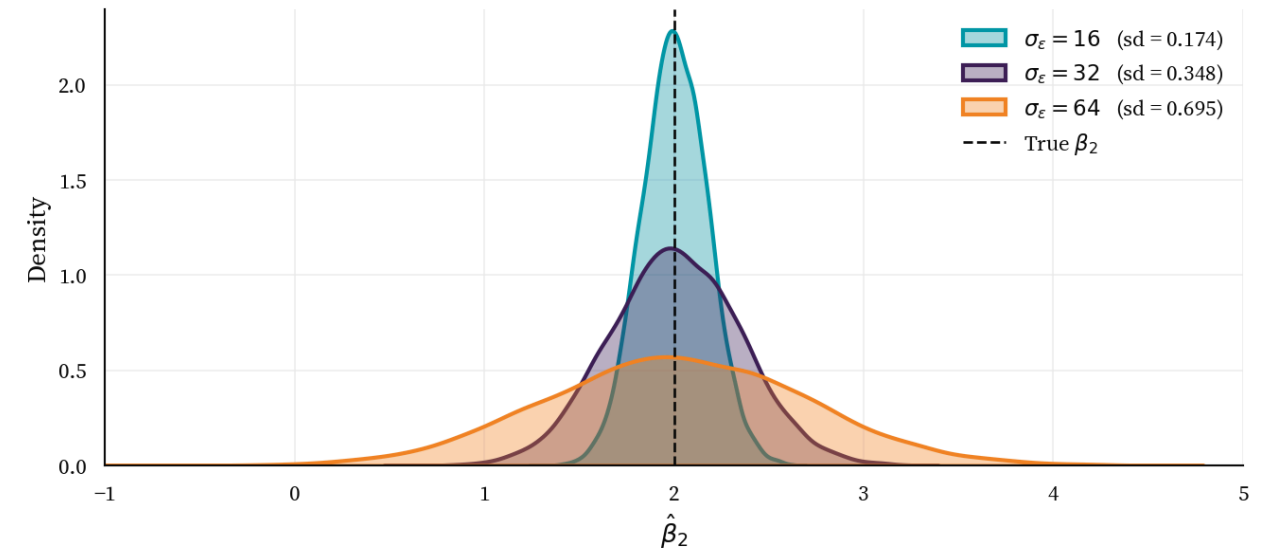
Element of the DGP	Effect on $\sum (x_{i2} - \bar{x}_2)^2 (1 - R_2^2)$	Distribution of $\hat{\beta}_2$
$\uparrow$ noise σ_ε	(numerator $\uparrow$)	more dispersed
$\uparrow$ sample size n	$\uparrow$ more terms in the sum	more concentrated
$\uparrow$ spread of x_2 ($U[0, a]$, larger a)	$\uparrow$	more concentrated
$\downarrow \sigma_v$ (more collinearity)	$R_2^2 \uparrow$, so $\downarrow$	more dispersed

1. More noise: σ_ε

CODE

```
1 results = {
2     r"$\sigma_\varepsilon = 16$": simulate_betas(sigma_eps=16),
3     r"$\sigma_\varepsilon = 32$": betas_base,
4     r"$\sigma_\varepsilon = 64$": simulate_betas(sigma_eps=64),
5 }
6 fig, ax = plt.subplots(figsize=(9, 4.2))
7 for label, betas in results.items():
8     sns.kdeplot(betas, fill=True, alpha=0.35, label=label, ax=ax)
9     ax.axvline(2, color="black", ls="--")
10 plt.show()
```

OUTPUT



Question

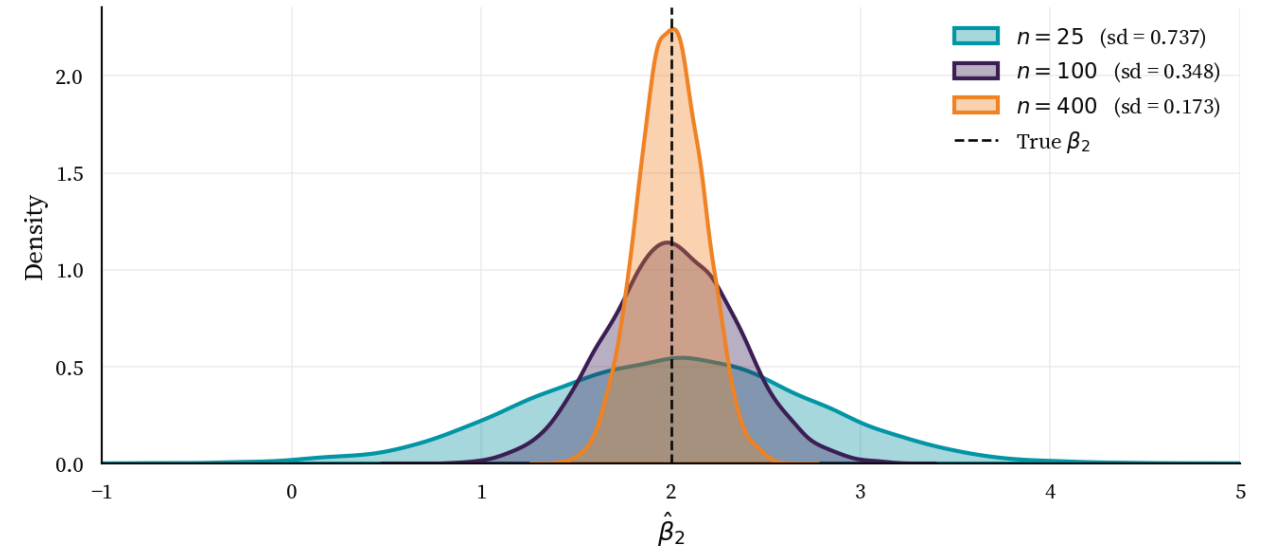
Doubling σ_ε : by how much does the sd of $\hat{\beta}_2$ change? Does the centre move?

2. More data: n

CODE

```
1 results = {
2     r"$n = 25$": simulate_betas(n=25),
3     r"$n = 100$": betas_base,
4     r"$n = 400$": simulate_betas(n=400),
5 }
6 fig, ax = plt.subplots(figsize=(9, 4.2))
7 for label, betas in results.items():
8     sns.kdeplot(betas, fill=True, alpha=0.35, label=label, ax=ax)
9 ax.axvline(2, color="black", ls="--")
10 plt.show()
```

OUTPUT



Question

Multiplying n by 4 divides the sd of $\hat{\beta}_2$ by ...? (Hint: $sd \propto 1/\sqrt{n}$.)

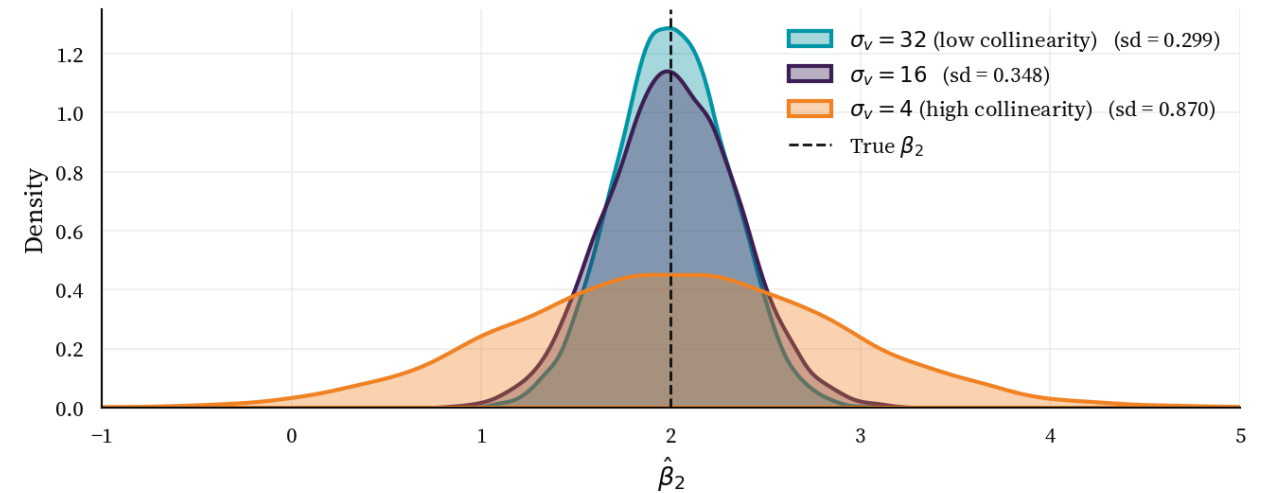
3. More collinearity: σ_v

Recall $x_{i3} = x_{i2} + v_i$: a **smaller** σ_v makes x_3 almost a copy of x_2 .

CODE

```
1 results = {
2     r"$\sigma_v = 32$ (low collinearity)": simulate_betas(sigma_v=32),
3     r"$\sigma_v = 16$": betas_base,
4     r"$\sigma_v = 4$ (high collinearity)": simulate_betas(sigma_v=4),
5 }
6 fig, ax = plt.subplots(figsize=(9, 3.8))
7 for label, betas in results.items():
8     sns.kdeplot(betas, fill=True, alpha=0.35, label=label, ax=ax)
9 ax.axvline(2, color="black", ls="--")
10 plt.show()
```

OUTPUT



Note

Collinearity does **not** bias $\hat{\beta}_2$: it only makes it imprecise, since there is little independent variation in x_2 left to learn from.

Your turn: play with the DGP

$$y_i = 4 + 2x_{i2} + 2x_{i3} + \varepsilon_i, \quad x_{i2} \sim U[0, a],$$

$$x_{i3} = x_{i2} + v_i$$

Each change re-runs **2,000** Monte Carlo replications.

$n = 100$

$\sigma_\varepsilon = 32$

$\sigma_v = 16$

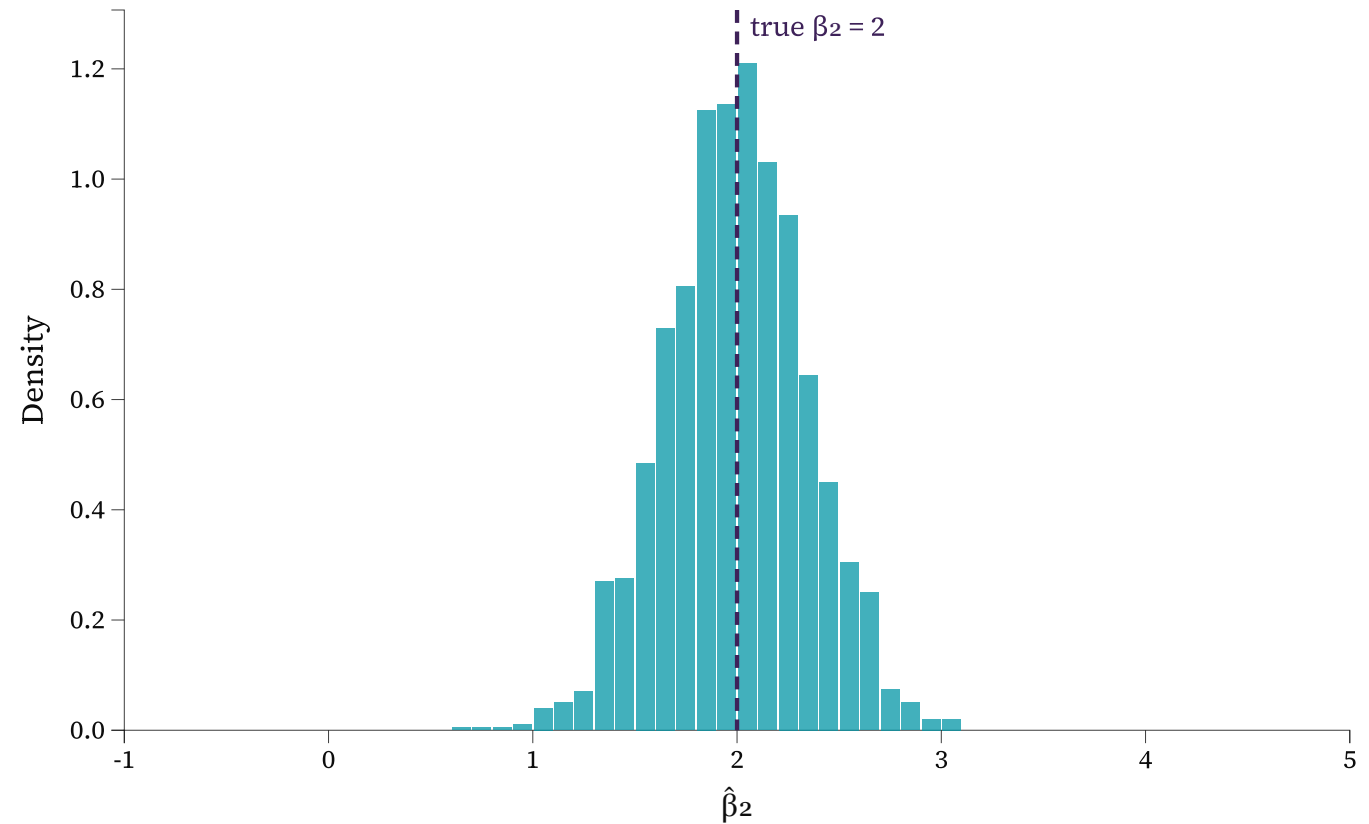
$a = 40$

Keep as reference

Re-run

Reset

Click **Keep as reference**, then move one slider: the **orange outline** keeps the old distribution.



mean($\hat{\beta}_2$) **1.995** · sd($\hat{\beta}_2$) **0.342** · theory $\approx$ **0.342**

Summary: concentrate or disperse?

Change in the DGP	Centre	Spread
$\sigma_\varepsilon \uparrow$	β_2	$\uparrow$ dispersed
$n \uparrow$	β_2	$\downarrow$ concentrated
range of $x_2 \uparrow$	β_2	$\downarrow$ concentrated
$\sigma_v \downarrow$ (collinearity $\uparrow$)	β_2	$\uparrow$ dispersed

Take-away

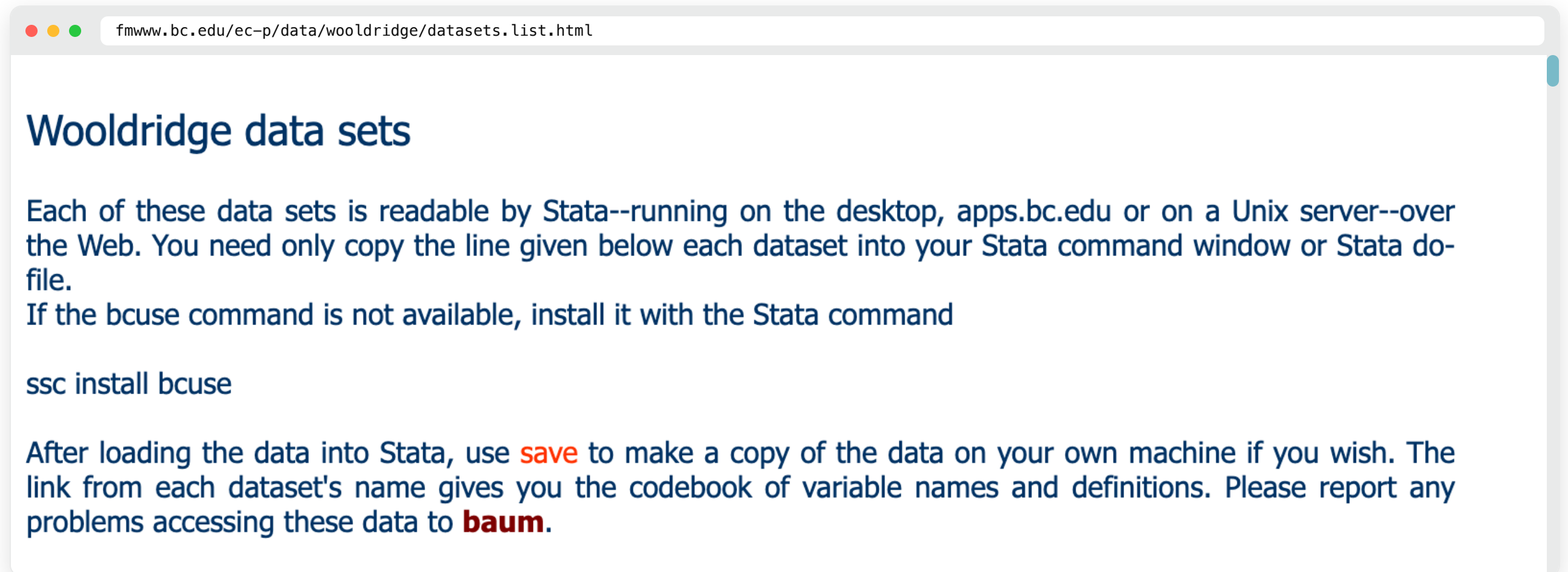
None of these changes moves the **centre**: as long as $E(\varepsilon \mid X) = 0$, OLS is unbiased.

They only change the **precision**, which is exactly what the *standard error* measures in a regression output.

A full regression output

Wooldridge datasets

The textbook by **Wooldridge** comes with **100+ real datasets**: wages, education, crime, housing, health, macro time series... All of them are available in Python through the `wooldridge` package (`description=True` prints the variables and the source):

A screenshot of a web browser window showing the Wooldridge datasets page. The address bar shows the URL 'fmwww.bc.edu/ec-p/data/wooldridge/datasets.list.html'. The page title is 'Wooldridge data sets'. The main text explains that each dataset is readable by Stata, either on a desktop, apps.bc.edu, or a Unix server. It provides instructions on how to use the 'bcuse' command in Stata to download the data. It also mentions that a codebook of variable names and definitions is available for each dataset and that users should report any problems to 'baum'.

fmwww.bc.edu/ec-p/data/wooldridge/datasets.list.html

Wooldridge data sets

Each of these data sets is readable by Stata--running on the desktop, apps.bc.edu or on a Unix server--over the Web. You need only copy the line given below each dataset into your Stata command window or Stata do-file.

If the bcuse command is not available, install it with the Stata command

```
ssc install bcuse
```

After loading the data into Stata, use **save** to make a copy of the data on your own machine if you wish. The link from each dataset's name gives you the codebook of variable names and definitions. Please report any problems accessing these data to **baum**.

Birthweight Data

The `bwght` dataset (Wooldridge) contains information on births in the United States, used to study the determinants of birth weight, in particular maternal smoking during pregnancy.

```
1 import wooldridge as woo
2
3 df = woo.dataWoo("bwght").dropna()
4 df[["bwght", "cigs", "parity", "faminc", "motheduc", "fatheduc"]].head(5)
```

	bwght	cigs	parity	faminc	motheduc	fatheduc
0	109	0	1	13.5	12.0	12.0
1	133	0	2	7.5	12.0	6.0
3	126	0	2	15.5	12.0	12.0
4	134	0	2	27.5	12.0	14.0
5	118	0	6	7.5	14.0	12.0

The model

$$\text{bwght}_i = \beta_1 + \beta_2 \text{cigs}_i + \beta_3 \text{parity}_i + \beta_4 \text{faminc}_i + \beta_5 \text{motheduc}_i + \beta_6 \text{fatheduc}_i + \varepsilon_i$$

where:

- `bwght`: birth weight, in **ounces**
- `cigs`: cigarettes smoked per day by the mother while pregnant
- `parity`: birth order of the child
- `faminc`: annual family income, in \$1000s
- `motheduc`, `fatheduc`: years of schooling of the mother and the father

Regression output in Python

```
1 import statsmodels.formula.api as smf
2
3 model = smf.ols("bwght ~ cigs + parity + faminc + motheduc + fatheduc", data=df).fit()
4 print(model.summary(slim=True))
```

OLS Regression Results						
=====						
Dep. Variable:	bwght		R-squared:	0.039		
Model:	OLS		Adj. R-squared:	0.035		
No. Observations:	1191		F-statistic:	9.553		
Covariance Type:	nonrobust		Prob (F-statistic):	5.99e-09		
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	114.5243	3.728	30.716	0.000	107.209	121.839
cigs	-0.5959	0.110	-5.401	0.000	-0.812	-0.379
parity	1.7876	0.659	2.711	0.007	0.494	3.081
faminc	0.0560	0.037	1.533	0.126	-0.016	0.128
motheduc	-0.3705	0.320	-1.158	0.247	-0.998	0.257
fatheduc	0.4724	0.283	1.671	0.095	-0.082	1.027
=====						

Anatomy of a regression table

For each coefficient k the output reports:

Column	Formula	Meaning
coef	$\hat{\beta}_k = \left[(X'X)^{-1} X'y \right]_k$	point estimate
std err	$se(\hat{\beta}_k) = \sqrt{\sigma^2 \left[(X'X)^{-1} \right]_{kk}},$ $\sigma^2 = \frac{\hat{\varepsilon}'\hat{\varepsilon}}{n-K}$	estimated sd of the sampling distribution
t	$t_k = \frac{\hat{\beta}_k - 0}{se(\hat{\beta}_k)}$	test of $H_0 : \beta_k = 0$
P> t	$2 P\left(t_{n-K} > t_k \right)$	two-sided p -value
[0.025, 0.975]	$\hat{\beta}_k \pm t_{n-K, 0.975} se(\hat{\beta}_k)$	95% confidence interval

Link with the Monte Carlo

The `std err` column is our best estimate, from **one** sample, of the spread we just saw across 10,000 simulated samples.

Computing it by hand: `cigs`

```
1 from scipy import stats
2
3 X = model.model.exog          # design matrix (with constant)
4 n, K = X.shape
5 XtX_inv = np.linalg.inv(X.T @ X)
6
7 sigma2_hat = model.ssr / (n - K)    # SSE / (n - K)
8 se = np.sqrt(sigma2_hat * np.diag(XtX_inv))
9 k = 1                               # position of cigs
10
11 t_stat = model.params.iloc[k] / se[k]
12 p_value = 2 * (1 - stats.t.cdf(abs(t_stat), df=n - K))
13 t_crit = stats.t.ppf(0.975, df=n - K)
14
15 print(f"beta = {model.params.iloc[k]:.4f}    se = {se[k]:.4f}    t = {t_stat:.3f}    p = {p_value:.2e}")
16 print(f"95% CI = [{model.params.iloc[k] - t_crit * se[k]:.4f}, {model.params.iloc[k] + t_crit * se[k]:.4f}]")
17 print(f"statsmodels: se = {model.bse.iloc[k]:.4f}    t = {model.tvalues.iloc[k]:.3f}    p = {model.pvalues.iloc[k]:.2e}")
```

```
beta = -0.5959    se = 0.1103    t = -5.401    p = 8.02e-08
95% CI = [-0.8124, -0.3794]
statsmodels: se = 0.1103    t = -5.401    p = 8.02e-08
```

Regression tables with `stargazer`

Why stargazer ?

`model.summary()` is great for **reading** results, but it is not what goes in a paper or an assignment.

stargazer

Builds a **publication-style** regression table from one or several fitted `statsmodels` models:

- coefficients with **significance stars**
- **standard errors** (or confidence intervals) in parentheses
- N , R^2 , adjusted R^2 , σ^2 , F
- rendered as **HTML** (notebook / slides) or **LaTeX** (assignment)

Exporting to LaTeX: `render_latex()`

```
1 from stargazer.stargazer import Stargazer
2
3 sg = Stargazer([model])
4 sg.title("Determinants of birth weight")
5 sg.table_label = "tab:bwght"      # for \ref{}
6 sg.rename_covariates({
7     "cigs": "Cigarettes per day",
8     "parity": "Birth order",
9     "faminc": "Family income",
10    "motheduc": "Mother's education",
11    "fatheduc": "Father's education",
12    "Intercept": "Constant",
13 })
14 sg.covariate_order(["cigs", "parity", "faminc",
15                    "motheduc", "fatheduc", "Intercept"])
16
17 latex = sg.render_latex()          # LaTeX code (a string)
18
19 with open("regression_output.tex", "w") as f:
20     f.write(latex)
```

Options: `sg.show_confidence_intervals(True)` (CIs instead of SEs),
`sg.significant_digits(2)`, `render_latex(only_tabular=True)`.

`regression_output.tex` (first lines):

```
\begin{table}[!htbp] \centering
  \caption{Determinants of birth weight}
  \label{tab:bwght}
\begin{tabular}{@{\extracolsep{5pt}}lc}
\\[-1.8ex]\hline
\hline \\[-1.8ex]
& \multicolumn{1}{c}{\textit{Dependent variable:}}
bwght}} \
\cr \cline{2-2}
\\[-1.8ex] & (1) \\
\hline \\[-1.8ex]
  Cigarettes per day &  $-0.596^{***}$  \\
& (0.110) \\
  Birth order &  $1.788^{***}$  \\
& (0.659) \\
  Family income &  $0.056^{\{ \}$  \\
& (0.037) \\
```

Using the table in your assignment

Put `regression_output.tex` next to your `main.tex` (in Overleaf: **Upload**), then:

```
1 \documentclass{article}
2 \begin{document}
3
4 As shown in Table~\ref{tab:bwght},
5 each additional cigarette per day
6 is associated with a lower birth weight.
7
8 \input{regression_output.tex}
9
10 \end{document}
```

As shown in Table 1, each additional cigarette per day is associated with a lower birth weight.

Table 1: Determinants of birth weight

	<i>Dependent variable: bwght</i>
	(1)
Cigarettes per day	-0.596*** (0.110)
Birth order	1.788*** (0.659)
Family income	0.056 (0.037)
Mother's education	-0.370 (0.320)
Father's education	0.472* (0.283)
Constant	114.524*** (3.728)
Observations	1191
R^2	0.039
Adjusted R^2	0.035
Residual Std. Error	19.789 (df=1185)
F Statistic	9.553*** (df=5; 1185)
Note:	*p<0.1; **p<0.05; ***p<0.01

The full output in LaTeX: t , p -values and CIs

`statsmodels` can export the **whole**
`summary()` to LaTeX:

```
1 latex_full = model.summary(slim=True).as_latex()
2
3 with open("regression_full.tex", "w") as f:
4     f.write(latex_full)
```

In `main.tex` (it needs `booktabs`):

```
1 \usepackage{booktabs} % in the preamble
2 ...
3 \begin{table}[!htbp]
4     \caption{Determinants of birth weight (OLS)}
5     \label{tab:full}
6     \input{regression_full.tex}
7 \end{table}
```

Table 1: Determinants of birth weight (OLS)

Dep. Variable:	bwght	R-squared:	0.039			
Model:	OLS	Adj. R-squared:	0.035			
No. Observations:	1191	F-statistic:	9.553			
Covariance Type:	nonrobust	Prob (F-statistic):	5.99e-09			
	coef	std err	t	P> t	[0.025	0.975]
Intercept	114.5243	3.728	30.716	0.000	107.209	121.839
cigs	-0.5959	0.110	-5.401	0.000	-0.812	-0.379
parity	1.7876	0.659	2.711	0.007	0.494	3.081
faminc	0.0560	0.037	1.533	0.126	-0.016	0.128
motheduc	-0.3705	0.320	-1.158	0.247	-0.998	0.257
fatheduc	0.4724	0.283	1.671	0.095	-0.082	1.027

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Preview: is `cigs` significant?

Exercise

Using the output above, test whether `cigs` is statistically significant at $\alpha = 5\%$.

1. State the **null and alternative hypotheses**.
2. Write the ***t*-statistic** and its distribution under H_0 .
3. State the **decision rule** and draw the acceptance and rejection regions (label the axes, include the critical values).
4. Draw the ***p*-value**. What is the minimum significance level at which `cigs` is significant?

Thanks!